ByteGraph-Dione: An Adaptive Dual-Format Graph Engine with Hotspot Awareness and Transaction Efficiency for Production-Scale Workloads

Chao Chen
ByteDance Inc.
Beijing, China
chenchao.chen@bytedance.com

Cheng Chen*
ByteDance Inc.
Singapore, Singapore
chencheng.sg@bytedance.com

Huiming Zhu
ByteDance Inc.
Chengdu, China
zhuhuiming.sy@bytedance.com

Shijiao Yang
ByteDance Inc.
Beijing, China
yangshijiao@bytedance.com

Yingqian Hu
ByteDance Inc.
Hangzhou, China
huyingqian@bytedance.com

Wei Wang
ByteDance Inc.
Beijing, China
wangwei.tab@bytedance.com

Wei Zhang
ByteDance Inc.
Beijing, China
zhangwei.95@bytedance.com

Bingyu Zhou
ByteDance Inc.
Chengdu, China
zhoubingyu.zby@bytedance.com

Miaomiao Cheng
ByteDance Inc.
Hangzhou, China
chengmiaomiao.123@bytedance.com

Lei Xiao
ByteDance Inc.
Hangzhou, China
xiaolei.ryan@bytedance.com

Ping Yang
ByteDance Inc.
Chengdu, China
yangping.deepdown@bytedance.com

Ye Wang
ByteDance Inc.
Beijing, China
wangye.tom@bytedance.com

Shuai Zhang
ByteDance Inc.
Beijing, China
zhangshuai.root@bytedance.com

Lei Zhang
ByteDance Inc.
San Jose, United States
zhanglei.michael@bytedance.com

Abstract

Modern social platforms from ByteDance (e.g., Douyin, Chinese version of TikTok) generate graph data at unprecedented scales, demanding cost-efficient and low-latency storage solutions. While our prior systems like ByteGraph [21][32] addressed these challenges through shared storage and workload-aware garbage collection, the growing convergence of Online Transactional Processing (OLTP) and Online Analytical Processing (OLAP) introduces new challenges to existing architectures. Specifically, OLTP operations require continuous updates on the latest data versions, whereas OLAP queries (e.g., under Snapshot Isolation) rely on stable versions for analytical consistency. This inherent tension forces traditional transaction mechanisms to maintain an excessive number of data versions, incurring additional access overhead. Meanwhile, rigid storage formats fail to adapt to divergent access patterns (exact queries vs. analytical scans), and hotspots caused by viral events or intensive graph analysis tasks further degrade performance in conventional coarse-grained replica architectures.

To address these limitations, we present ByteGraph-Dione, a graph database engine optimized for dynamic hybrid workloads. Our system introduces three key innovations: First, a lightweight transaction protocol that optimizes OLTP and OLAP concurrency control to minimize the overhead of transaction support. Secondly, we propose an adaptive dual-format storage engine. It dynamically organizes data into row-oriented structures to handle exact queries from OLTP, and columnar layouts for efficient analytical scans from OLAP, thereby optimizing overall access performance. Third, we propose a fine-grained scaling mechanism that creates replicas based on hotspot access patterns and efficiently distributes the traffic, thereby avoiding the additional CPU and memory overhead associated with creating full read-only replicas. Experimental evaluations on ByteDance workloads show that ByteGraph-Dione improves end-to-end performance and resource efficiency by large margins.

CCS Concepts

• **Information systems** → **Data management systems**; **Storage management**.

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

SIGMOD Companion '26, Bengaluru, India.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2450-3/2026/05
https://doi.org/10.1145/3788853.3803073

Keywords

Graph Database; Cloud Storage; Graph Storage

ACM Reference Format:

Chao Chen, Cheng Chen, Huiming Zhu, Shijiao Yang, Yingqian Hu, Wei Wang, Wei Zhang, Bingyu Zhou, Miaomiao Cheng, Lei Xiao, Ping Yang, Ye Wang, Shuai Zhang, and Lei Zhang. 2026. ByteGraph-Dione: An Adaptive Dual-Format Graph Engine with Hotspot Awareness and Transaction Efficiency for Production-Scale Workloads. In *Companion of the International Conference on Management of Data (SIGMOD Companion '26), May 31–June 5, 2026, Bengaluru, India.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3788853.3803073

1 Introduction

In ByteDance's applications, such as Douyin (the Chinese version of TikTok), graph data plays a pivotal role across diverse business scenarios. These platforms generate billions of nodes and hundreds of billions of edges daily, presenting significant challenges in managing large-scale graph data in terms of cost and performance. Existing graph databases fail to meet ByteDance's stringent requirements for both storage volume and query throughput. To address these challenges, we proposed ByteGraph [21][32], a distributed graph database that employs a two-tier architecture. As shown in Figure 1, ByteGraph consists of Execution Layers and Storage Engine Layer. This architecture decouples the query executor from the persistent storage layer, allowing for independent scalability and optimization. The storage engine is built on an append-only, cost-effective shared storage solution. ByteGraph organizes data using a BW-Tree and employs a leader-follower synchronization mechanism to ensure consistency for large-scale real-time graph analytics.

As ByteDance's business continues to grow, we have observed a rapid shift in graph data workloads. Increasingly, more business scenarios require handling both Online Transaction Processing (OLTP) and Online Analytical Processing (OLAP) workloads on the same graph. This demand arises from applications that seek to support real-time transactional processing while simultaneously enabling complex analytical queries on a unified data platform. For example, in Douyin's "like" scenario, clicking the button necessitates storing the user-video interaction and then executing graph algorithms on the updated data to gain insights into user behavior or trending topics. When handling production-scale workloads, this integration of OLTP and OLAP workloads places stringent demands on the ByteGraph system.

First, OLTP workloads on graph data typically involve simple operations such as adding, deleting, or updating specific edges/nodes, or retrieving partial attributes of immediate neighbors. In contrast, OLAP queries demand complex relational analyses (e.g., tracing whether a user's video was propagated by followers). These divergent access patterns—exact key-value queries for OLTP versus full-scope scans for OLAP—create significant challenges for adjacency list storage. Traditional single-format storage systems (e.g., row-oriented in prior ByteGraph) struggle to balance efficiency for both workloads, especially in scenarios requiring high queries per second (QPS) for mixed operations under strict cost constraints.

Moreover, to ensure the stability and correctness of OLTP and OLAP results, applications require the graph database to support at least Snapshot Isolation (SI) level transactions while handling mixed OLTP and OLAP workloads. This process inherently causes each vertex and edge to accumulate multiple versions. However, traditional SI implementations generate excessive Multi-Version Concurrency Control (MVCC) historical versions when processing mixed workloads, consuming substantial memory and degrading query performance. Additionally, OLTP operations such as user likes and comments impose strict millisecond-level latency constraints. Consequently, we must design a lightweight transaction path that keeps SI for long OLAP scans yet limits overhead for latency-sensitive OLTP—a difficult design problem in practice.

Finally, data hotspots are a frequent occurrence. These are typically triggered by high-traffic events on social platforms, such as a viral video (e.g., a breaking news clip) or a major trending topic (e.g., a sports finals), as both scenarios first generate a surge of OLTP operations (e.g., likes, shares) and are immediately followed by OLAP tasks to analyze the burst of new interactions. These scenarios lead to sharp increases in access to certain nodes in the graph database. Traditional methods for handling hotspots typically involve creating full replicas of the data shard experiencing high traffic, allowing the original traffic directed to the shard to be evenly distributed across multiple replicas. However, ByteDance's workload presents two key challenges with this approach. First, constructing such data replicas requires copying a large amount of data, imposing significant network and memory consumption. Second, limited cache capacity plus shifting hotspots and generic caching policies often yield low hit rates, so operators must deploy still more replicas merely to sustain hotspot QPS.

To address the challenges posed by dynamic workloads, we enhanced the storage engine of ByteGraph and introduced ByteGraph-Dione (**D**ual-format, h**O**tspot-aware, tra**N**saction-efficient graph **E**ngine). The system is built upon ByteGraph and tailored to the challenges of real-world, production-scale workloads. Our key contributions are as follows:

- **In-depth Analysis of Industrial Graph Workloads**: We conducted a comprehensive analysis of industrial-grade graph workload characteristics in real business scenarios and explored the performance challenges.
- **Lightweight Transaction**: ByteGraph-Dione introduces a lightweight transaction mechanism tailored for mixed OLTP and OLAP scenarios. This mechanism retains the historical versions required for short transactions entirely in memory while aggregating long-running OLAP transactions to reduce the number of data versions. This design effectively minimizes the demand for querying historical versions and reduces memory consumption, thereby mitigating the potential performance impact of MVCC.
- **Adaptive Dual Storage Format**: Building upon the row-based storage of the previous ByteGraph, we propose a columnar storage format integrated with the BW-Tree. Leveraging a row-column cost model derived from historical workload statistics, ByteGraph-Dione dynamically adapts the storage format for each adjacency list based on the varying intensities of OLTP and OLAP workloads. This adaptive approach not only optimizes performance for complex and dynamic workloads but also eliminates redundant storage.

- **Fine-Grained Scaling Mechanism**: We implemented a fine-grained scaling mechanism that integrates lazy hot data movement at the adjacency list level with a fine-grained distribution mechanism for hotspot data access. This mechanism significantly improves ByteGraph-Dione's performance in hotspot scenarios.

2 Background

2.1 HTAP Relation Database

OLTP systems (e.g., MySQL [3], PostgreSQL [4]) use row-based storage for transactional efficiency, while OLAP systems (e.g., Presto [27]) favor columnar storage for analytics. Traditional separation necessitates ETL processes, compromising data freshness. Hybrid Transactional/Analytical Processing (HTAP) databases solve this by supporting concurrent transactions and real-time analytics. They are categorized by storage: unified-format systems (e.g., COLIBRI [26], SAP HANA [15], SingleStore [25]) merge row and columnar storage in one engine, ensuring consistency but potentially impacting OLTP performance; dual-format systems (e.g., Oracle Dual [19], SQL Server CSI [20]) use redundant storage for freshness and immediacy, but may lack performance isolation; distributed systems (e.g., PolarDB-IMCI [29], F1-lighting [31], TiDB [18], MySQL HeatWave [2], and ByteHTAP [13]) employ asynchronous replication for better isolation, albeit with a trade-off in data freshness.

2.2 HTAP On Graph Database

Existing graph databases exhibit distinct trade-offs between OLTP and OLAP workloads. Systems such as JanusGraph [1], ArangoDB [5], OrientDB [6], and Titan [7] target OLTP workloads but rely on non-native storage backends, leading to inefficient traversals. Neo4j offers high traversal efficiency yet is constrained by its single-node design, while A1 [12] focuses on in-memory processing but lacks scalability. TAO [11] stores graph data in MySQL and improves performance via caching, but still falls short of native graph databases in both efficiency and usability. For analytical (OLAP) workloads, systems such as Pregel [24], GraphLab [23], PowerGraph [16], GraphX [17], and GraphScope [30] provide efficient distributed in-memory computation, yet lack transactional support required by OLTP.

HTAP graph databases aim to unify OLTP and OLAP processing. TigerGraph [14] achieves high performance by keeping all data in memory, but its memory footprint limits scalability for ByteDance-scale workloads. Galaxybase [28] optimizes traversals and single-edge queries via its Edge-Page format but suffers from row-store bottlenecks in large-scale analytics and MVCC overhead under high-throughput OLTP transactions.

2.3 Architecture of the Previous ByteGraph

As shown in Figure 1, ByteGraph consists of two primary layers: the execution engine layer and the storage engine layer. The execution engine layer is responsible for parsing, optimizing, and executing graph query languages. The storage engine layer is built on top of shared storage that provides an append-only interface, organizing graph data into a graph-native format to efficiently support graph data retrieval and updates. In this paper, we focus on optimizing

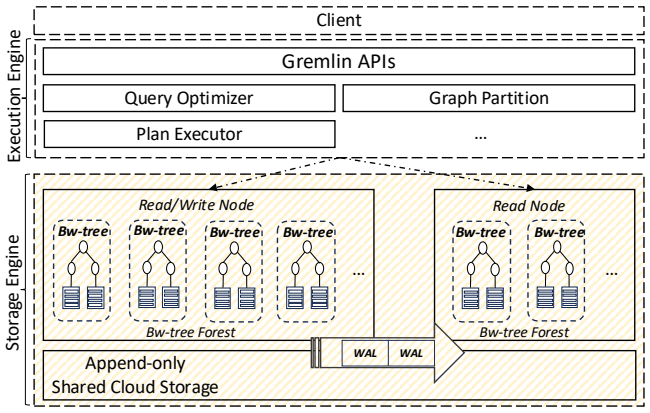


Figure 1: The Architecture of Previous ByteGraph

the storage engine layer. As shown in the lower half of Figure 1, the previous ByteGraph [32] proposed to employ a BW-Tree forest as its indexing structure. ByteGraph supports scalable real-time analytics through a shared-storage–based read–write synchronization mechanism, leveraging write-ahead logging to ensure strong consistency between leader and follower replicas.

As business workloads grow in complexity and diversity, there is increasingly a need to simultaneously manage multiple OLTP and OLAP workloads on the same graph database. With the increasing prevalence of OLAP workloads, the frequent emergence of hotspots in online systems has become a significant issue. Additionally, increasingly stringent requirements have been set for transaction isolation among different workloads within the same graph database. Serving fast, dynamic OLTP and OLAP access under tight cost budgets therefore strains the next generation of ByteGraph storage design.

3 Motivation

3.1 Real-world Graph Workloads

3.1.1 Workload Categories. As shown in Table 1, OLTP queries handle basic graph operations, such as inserting, updating, or fetching a specific node/edge and its immediate neighbors. In contrast, OLAP queries perform complex multi-hop relational analyses, like multi-layer friend exploration or advanced path-based inquiries to uncover indirect influence and propagation patterns. OLAP queries typically exhibit long latencies, ranging from 1 second to 30 minutes. Meanwhile, given that OLTP operations primarily involve single-hop queries, they require very strict latency control, typically necessitating responses in under one second. Furthermore, most OLAP queries require Snapshot Isolation (SI) to ensure consistency, whereas OLTP workloads typically rely on the lighter Read Committed (RC) level. Given OLTP's stringent latency demands and the limited need for SI in this context, a more lightweight transaction processing approach is essential. The representative Gremlin queries for OLTP and OLAP operations are available on GitHub[1].

3.1.2 Hybrid Workloads. As ByteDance's business evolves, single datasets often handle multiple workloads (Table 2). In social interaction scenarios, about 85%–95% are OLTP workloads (e.g., likes

[1]https://github.com/BiterrorChen/ByteGraph-Dione-Query-Set/tree/master/

Workload	Overall Thpt (QPS)	Query Pattern	Latency	Transaction Isolation
OLAP	hundreds of thousands (10^5)	2+ Hops, PathQuery	up to 30min (Latency insensitive)	Snapshot Isolation
OLTP	billions (10^9)	1 Hop	< 1s	Read Committed

Table 1: OLAP/OLTP Workloads

Applications	Description	OLAP(%)	OLTP(%)
Social interaction	Douyin likes, comments and other interactive data	5 ~ 25	75 ~ 95
Douyin relationship	Douyin relationship data	10 ~ 35	65 ~ 90
Data lineage	Relationship between offline Hive tables, data tasks, and data features	70 ~ 90	10 ~ 30

Table 2: Real-world Graph Applications

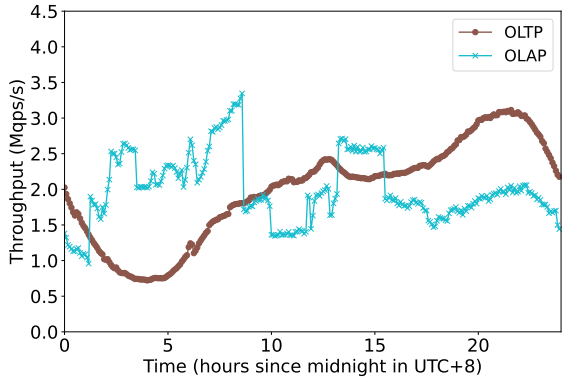


Figure 2: The Throughput of OLTP and OLAP in Douyin Cluster

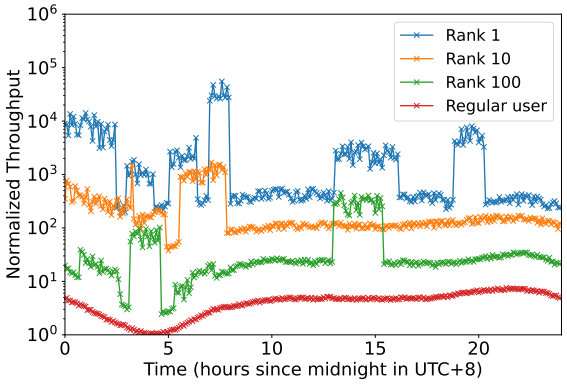


Figure 3: Comparison of Hotspot Changes Between Top Users and Regular Users in a Day

and comments) and 5%–15% are OLAP, reflecting dual demands for real-time processing and analysis. In Douyin's relationship data, 65%–90% are OLTP and 10%–35% are OLAP, supporting real-time user relationships and behavior-based recommendations. For data lineage, 10%–30% are OLTP and 70%–90% are OLAP, focusing on data quality and traceability analysis. These scenarios indicate significant variability in the proportion of OLAP and OLTP workloads across different business domains, and those demands shift with product features and traffic patterns hour by hour. Figure 2 illustrates the dynamic daily access patterns within a Douyin social graph cluster, revealing two key observations. First, the ratio of OLTP to OLAP workloads fluctuates significantly over time. Second, at any given moment, different users' adjacency lists experience substantial variations in load. For instance, when an influencer posts a new video, it triggers a surge of OLTP queries (e.g., numerous followers viewing his profile and liking the video). Concurrently, the system may be executing heavy OLAP scans on other influencers' data (e.g., to trace the propagation path of a recent trend), so workload type and intensity can differ sharply from one adjacency list to the next.

3.1.3 Access Hotspot. As shown in Figure 3, we observed the frequent occurrence of sudden hotspot phenomena, which often result in adjacency list hotspots and impact overall performance. Surges in OLAP traffic, driven by multi-hop analysis tasks involving deep scans and computations, exacerbate the problem. This is because hotspot data often concentrates on a small number of adjacency lists, following the power-law distribution typical of graph data [10]. Figure 3 illustrates throughput variations over a day in the Douyin relationship scenario, highlighting three key observations: (1) Hotspots often emerge simultaneously across multiple adjacency lists because highly connected influencer nodes attract numerous queries, especially during multi-hop analysis tasks. (2) Hotspot intensity varies significantly, with some nodes experiencing query volumes several times or even tens of times higher than others, as shown in the comparison of users ranked 1st, 10th, 100th, and average users. (3) Hotspots exhibit persistence, remaining active for at least half an hour rather than dissipating quickly. These traits complicate hotspot routing and cache planning for graph stores and call for designs that tolerate sustained read spikes on a few hot lists.

3.2 Challenges in System Design

3.2.1 Performance Degradation from Transaction Support in Hybrid OLAP & OLTP Workloads. As shown in Table 1, both OLAP and OLTP workloads in graph databases require transaction support, typically implemented through Multi-Version Concurrency Control (MVCC). MVCC provides each transaction with a versioned snapshot of data at the granularity of individual tuples, ensuring isolation and a consistent view during execution. However, the characteristics of graph workloads often result in excessive historical versions. Figure 4 illustrates this issue: updates to Edge1 three

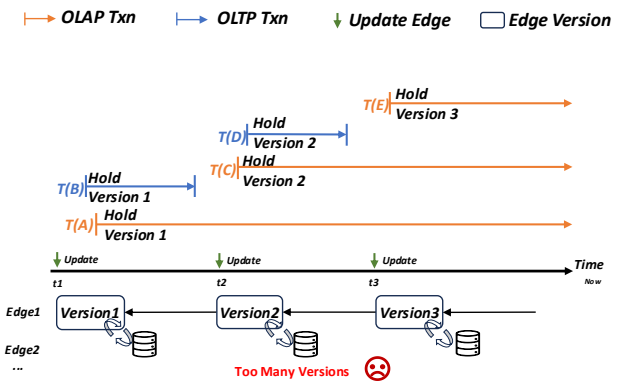


Figure 4: Challenges of Transaction Support in Hybrid OLAP & OLTP Workloads

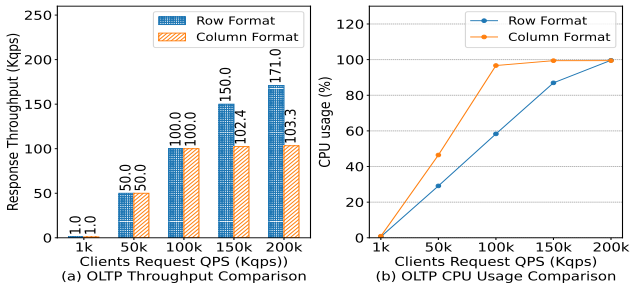


Figure 5: Compare the Throughput and CPU Utilization for Row-format and Column-format Using a Typical OLTP Workload

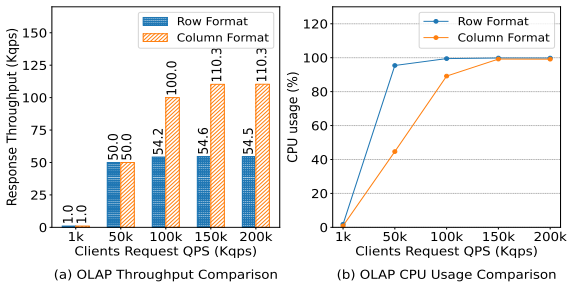


Figure 6: Compare the Throughput and CPU Utilization for Row-format and Column-format Using a Typical OLAP Workload

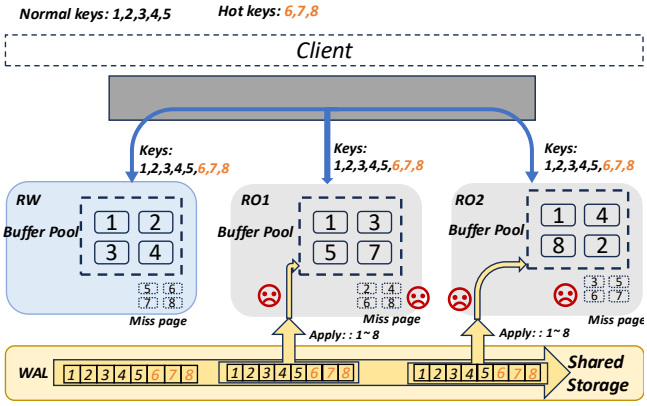


Figure 7: Issues with Traditional Hotspot Replica Solutions

times from t1 to t3 thus generate three versions, with interleaved OLAP long-running transactions (e.g., T(A), T(C), T(E)) and OLTP short transactions (e.g., T(B), T(D)). Two key challenges arise: (1) Although short OLTP transactions T(B) and T(D) have brief execution times, the data versions (Version1 and Version2) they use cannot be released immediately, as they are still needed by the long-running OLAP transactions T(A) and T(C). (2) OLAP tasks, which can run for up to 30 minutes and involve extensive wide-column scans, require maintaining a large number of versions in memory. Bursty updates and heterogeneous OLAP jobs make the problem worse in production, so engines must bound version retention when OLTP and OLAP contend for the same adjacency lists at once.

In mixed OLTP and OLAP environments, performance degradation mainly arises from two factors: first, disk I/O amplification due to frequent reads and writes of historical versions exceeding memory capacity; second, CPU overhead from traversing long version chains for snapshot retrieval. In the Douyin relationship scenario, high update throughput (hundreds of thousands of QPS) quickly produces numerous historical versions, some lasting tens of minutes because of prolonged OLAP executions. Together, these effects argue for low-overhead isolation and aggressive version recycling wherever OLTP and OLAP run on shared graph partitions.

3.2.2 The Need for Adaptive and Fine-Grained Multi-Format Storage Engine. In the Douyin relationship scenario, we evaluated the performance of OLTP queries (e.g., retrieving the 10 most recent edges of a node's neighbors) and OLAP queries (e.g., retrieving all first-degree neighbors of a node). Figures 5 and 6 show that at low QPS, CPU consumption differences between row and column formats are minimal. However, as QPS increases, differences become pronounced: row format achieves 1.65× higher throughput for OLTP queries, while column format delivers 2.02× better throughput for OLAP workloads. Currently, previous ByteGraph uses row format for adjacency lists, which excels at OLTP queries but struggles with OLAP workloads requiring large-scale scans. Traditional databases demonstrate that row storage suits OLTP workloads, while column storage better handles OLAP workloads. Dual-format adjacency lists are therefore attractive: one cluster can reach million-scale QPS while combining point-style OLTP access and wide OLAP scans in one deployment.

To balance performance and cost for ByteDance's dynamic workloads, traditional HTAP databases adopt dual-format storage but incur significant redundancy and storage costs, especially for smaller adjacency lists with low throughput. A one-size-fits-all strategy is ineffective given the power-law distribution of graph data and access patterns. Fine-grained adaptive layouts are needed to cap bytes while still feeding skewed OLTP/OLAP mixes in large interactive graph services.

3.2.3 Managing Burst Hotspot Issues in Adjacency List Access. OLAP workloads in graph databases often experience rapid fluctuations and sudden hotspots, causing certain nodes to face intense read pressure. Traditional cloud-native databases like PolarDB and

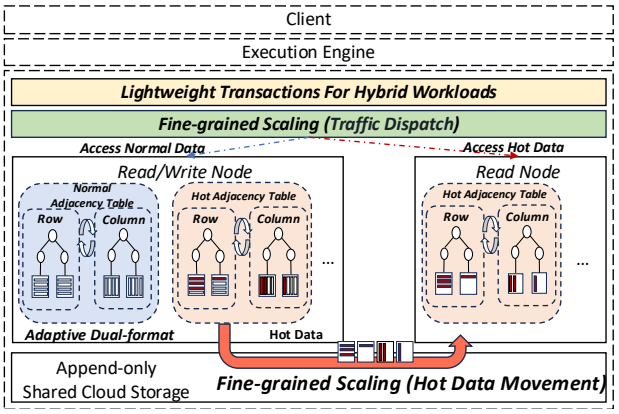


Figure 8: The Architecture of ByteGraph-Dione

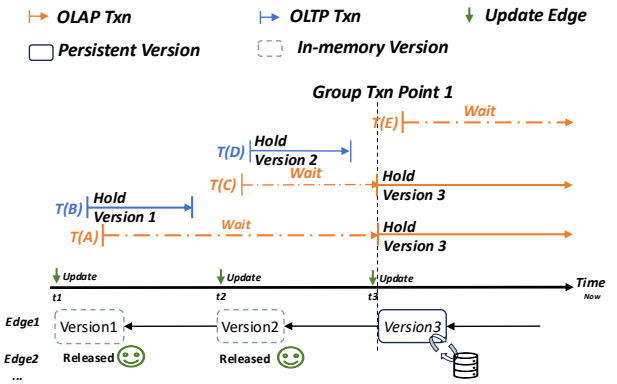


Figure 9: Lightweight Transactions For Hybrid Workloads

Aurora address this by adding in-memory read-only replicas, which distribute the load but introduce new challenges. Firstly, creating full data replicas across multiple nodes incurs high CPU costs due to log replay and data reconstruction. As shown in Figure 7, addressing hotspots (e.g., Keys 6, 7, 8) requires read-only nodes to construct pages for all keys, including non-hotspot ones (e.g., Keys 1–5), increasing resource overhead. Secondly, workloads are evenly distributed across nodes, forcing each to cache the entire dataset. This pattern wastes RAM and yields frequent cache misses when buffer pools are small (e.g., four pages per node), which hurts tail latency most under hotspot reads.

4 Design and Implementation

4.1 System Overview

Similar to [32], ByteGraph-Dione adopts a compute-storage separation architecture, using a BW-Tree engine on append-only cloud storage. To ensure snapshot isolation for hybrid workloads, we implemented a "Lightweight Transactions for Hybrid Workloads" mechanism (4.2) that minimizes performance impact. By introducing row and column storage formats and an "Adaptive Dual Storage Format" (4.3.3) based on a simplified cost model, we dynamically optimize storage under changing workloads. Additionally, our "Fine-grained Scaling" mechanism selectively replicates hotspot data to raise cache hit rates and steer query traffic more evenly (Section 4.4).

4.2 Lightweight Transactions

As described in 3.2.1, the core challenge lies in minimizing the performance impact of implementing snapshot isolation. We observed two phenomena from the workload analysis presented in Section 3.1: (1) OLTP transactions prioritize low latency, while most OLAP transactions tolerate higher latency. (2) As shown in Table 1, OLAP transactions are significantly more complex compared to OLTP transactions, with execution times reaching up to 30 minutes. Thus, OLAP requires maintaining multiple data versions over extended periods to support continuous analysis. In contrast, most OLTP transactions have short execution times and small data volumes, thus they do not require maintaining many snapshots. Motivated by these observations, we adopt the following implementation approach.

4.2.1 Aggregation for long-running OLAP Transactions. To implement MVCC on ByteGraph-Dione, our version management strategy generates a new data version for each update operation. As shown in Figure 4, Edge1 has three versions, with version 3 being the latest and version 1 the oldest. To address excessive versioning caused by long-running OLAP transactions, we propose an aggregation-based execution strategy. Transactions likely to be long-running, such as those with more than two-hop traversals, are identified and executed using batch aggregation. For example, as shown in Figure 4, traditional processing would generate multiple versions of Edge1 for T(A) and T(C). Instead, we aggregate these transactions within the same interval (e.g., t1 to t3), requiring only version 3 to be persisted, thereby reducing memory consumption as well as disk I/O pressure. We also observed that longer aggregation intervals further reduce the amount of stored historical data but may increase OLAP latency and reduce accuracy. In Section 5.3, we explore how to dynamically adjust the interval threshold based on workload patterns to determine the optimal aggregation interval. To prevent excessive memory consumption by long-running OLAP transactions, the versions required for these transactions are marked as persistent. When memory becomes constrained, persistent versions can be swapped to storage, preventing overflow and ensuring stable execution.

4.2.2 In-memory MVCC for short OLTP Transactions. OLTP transactions typically have short execution times. To meet their strict low-latency requirements, these transactions are executed immediately upon arrival (T(B) and T(D) in Figure 9). For in-memory version management, aside from the shared data versions used by long-running OLAP transactions, the data used by OLTP transactions generates independent in-memory versions (e.g., version 1 and version 2). These in-memory versions are stored exclusively in memory because OLTP transactions have short execution times and strict low-latency requirements. Keeping these accessed data versions in memory significantly reduces the time overhead associated with retrieving historical data swapped out to storage.

4.2.3 In-memory Space Reclamation. As described in Section 4.2.1, the delay and batch execution of long-running OLAP transactions result in a large amount of in-memory version data being held only by OLTP transactions during the intervals between these batch

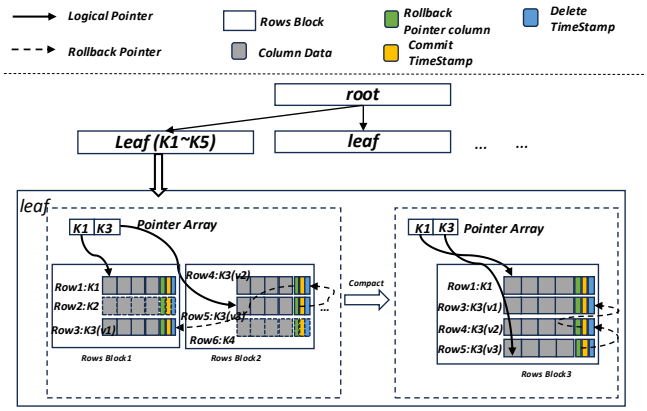


Figure 10: Row Format

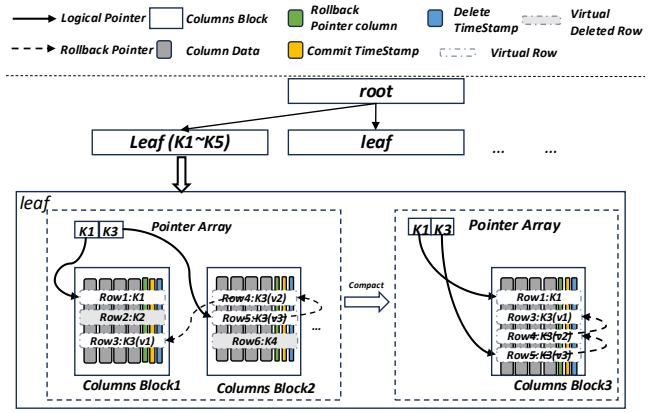


Figure 11: Column Format

executions (version1 and version 2). Once these short transactions are completed, this version data is no longer needed. To prevent these historical versions from occupying memory for extended periods, we set a time threshold to control the retention of in-memory versions. In production environments, this threshold is set to 30 seconds by default, which is sufficient to cover the execution time of all OLTP transactions involving two-hop queries or less. Through this approach, we effectively reduce the number of MVCC data versions in memory, shorten version chains, and speed up version-chain walks during queries.

4.3 Adaptive Dual Storage Format

4.3.1 Dual-Format Graph Storage Engine. In Section 3.1, it has been demonstrated that row storage is well-suited for OLTP workloads, while column storage is better suited for wide range scans. This section explains how ByteGraph-Dione implements these two storage formats using the BW-Tree forest. In previous ByteGraph [32], we used a vertex-centric adjacency-list layout: each vertex is mapped to one BW-Tree, and its outgoing edges are sorted by edge key and stored in leaf pages, with edge/vertex attributes stored contiguously.

In the previous ByteGraph [32] BW-Tree engine, each LeafPage stored data in row format. As illustrated in Figure 10, using Douyin's "like" scenario as an example, a user's BW-Tree stores information about the videos they have liked. The video IDs are used as the BW-Tree keys, and the LeafPage contains a batch of video records (K1 K4). Each video's details, such as category, duration, and other attributes, are stored in row format within the Leaf Node.

For implementing columnar storage in pages, outstanding solutions such as PAX [9] already exist. PAX divides a page into multiple sub-relations, leveraging data affinity in the cache to deliver excellent point-query performance. However, implementing a columnar format in ByteGraph faces unique challenges. The column format must be adapted to fit within the constraints of BW-Tree's append-only architecture. It must also maintain efficient row-scanning capabilities in column storage to support frequent OLTP queries effectively. Additionally, the design must handle insert, delete, and update operations seamlessly on BW-Tree while ensuring high performance for OLAP queries. As shown in Figure 11, the ColumnsBlock organizes row attributes by columns, similar to the storage format of Apache Arrow [8]. A page contains multiple ColumnsBlocks, with each update generating a new block while old blocks remain read-only. Periodic compaction optimizes storage space and scanning performance. A pointer array is used to locate specific rows within the ColumnsBlock. Each logical pointer consists of a block identifier (ID) and an offset, which can be used to retrieve the complete row through address computation.

Although the LeafPage supports two formats, its fundamental structure within the LeafPage is similar, primarily consisting of a Pointer Array and Blocks (Row Blocks or Column Blocks). Its data query interface offers two key features: row scanning with filtering and column scanning capabilities. Additionally, it supports a multi-version storage mechanism and transactional multi-version concurrency control. The following examples demonstrate how the same OLTP and OLAP instructions are executed under the two storage formats.

Case 1: Executing an OLTP query to retrieve all attributes of K3:

- **Row Format:** The BW-Tree locates the leaf page corresponding to K3. The Pointer Array is then used to identify the latest version, Row5, which is evaluated against MVCC requirements. If an older version is needed, the version chain is traversed to retrieve the appropriate historical version.
- **Column Format:** Similar to the row format, the BW-Tree locates the leaf page corresponding to K3. The Pointer Array identifies the latest version, Row5, and traverses the version chain to find the suitable version. Unlike the row format, attributes in Row5 are stored non-contiguously in memory due to the columnar layout.

Case 2: Retrieving all users who liked a video after a specific date. For simplicity, we focus on accesses within a single leaf page; accesses to other pages follow a similar process.

- **Row Format:** Traverse the Pointer Array to fetch each row (finding the version that satisfies MVCC requirements by following the version chain). Parse the row to extract the like timestamp attribute and evaluate whether it meets the filtering condition.
- **Column Format:** Traverse different Column Blocks to scan the "like timestamp," "commit timestamp," and "delete timestamp" columns. Retrieve data that meets the filtering

condition, ensuring the "commit timestamp" and "delete timestamp" are used to select versions that satisfy MVCC requirements and exclude logically deleted data.

Since the data is organized in an append-only manner, periodic garbage collection is performed by compacting multiple old blocks into a new one.

4.3.2 Online Row-Column Format Conversion. To adapt to the dynamic changes of hybrid workloads, we employ multiple storage formats to optimize performance. However, maintaining both formats simultaneously increases storage costs. To address this, we implemented an adjacency list-level storage format conversion mechanism that supports online format conversion, allowing the system to select the optimal storage format for the specific workloads faced by different adjacency lists.

The following outlines the process of format conversion, using the introduction of a new column format as an example. The process consists of five stages:

(1) **Appending new data:** The latest data for the adjacency list is recorded in a dedicated append-only page.
(2) **Snapshotting row storage:** A real-time snapshot of the row-stored adjacency list is captured.
(3) **Converting to column format:** The snapshot of the row-stored adjacency list is converted and stored in the new column format.
(4) **Applying incremental updates:** The incremental data from the first stage is applied to the newly converted adjacency list in column format.
(5) **Dual-format synchronization:** Once most of the incremental data pages have been replayed, the system begins synchronously writing data to both the row and column storage formats.

The entire conversion process does not interrupt service. To minimize performance fluctuations in graph data access during conversion, we control the granularity of the format conversion at the individual adjacency list level. Furthermore, we evaluated the overhead of row-column conversions in Section 5.4.2.

4.3.3 Row-Column Conversion Cost Function. We collected workload traces from multiple ByteDance production scenarios, covering page sizes from a few hundred bytes to tens of kilobytes. The datasets include diverse OLTP queries (e.g., Top-k scans, single-edge lookups, and attribute filters) and OLAP queries accessing 1–20 columns with filtering conditions. By recording the number of row and column attributes accessed and benchmarking performance under both row and column layouts, we derived the following cost function.

$$\begin{aligned}
\text{Cost}_{\text{row}} &= \sum_{i=1}^{\mathbf{Q}} (\mathbf{a}_{\text{row}} \times \mathbf{N}_{\text{rows}_i} \times \mathbf{N}_{\text{attr}_i} + \mathbf{B} \times \mathbf{N}_{\text{rows}_i} + \mathbf{C}_{\text{row}}) \\
\text{Cost}_{\text{col}} &= \sum_{i=1}^{\mathbf{Q}_{\text{olap}}} (\mathbf{a'}_{\text{col,olap}} \times \mathbf{N}_{\text{rows}_i} \times \mathbf{N}_{\text{attr}_i} + \mathbf{B} \times \mathbf{N}_{\text{rows}_i} + \mathbf{C'}_{\text{col,olap}}) \\
&\quad + \sum_{i=1}^{\mathbf{Q}_{\text{oltp}}} (\mathbf{a'}_{\text{col,oltp}} \times \mathbf{N}_{\text{rows}_i} \times \mathbf{N}_{\text{attr}_i} + \mathbf{B} \times \mathbf{N}_{\text{rows}_i} + \mathbf{C'}_{\text{col,oltp}}) \quad (1)
\end{aligned}$$

Considering the high timeliness requirements for calculating the cost function online, we simplified the cost module. Page location, cache effects, and I/O overhead are unified into a constant **C**. Additionally, the cost of binary search for locating data rows within a page and retrieving the appropriate MVCC version is represented as a constant **B**.

Let us first examine the cost function for the row format. As mentioned in Section 5.2.1, each request, whether OLTP or OLAP, involves the following steps:

- **Step 1: Locate the BW-Tree Leaf Node.** The query locates the relevant leaf node in the BW-Tree, with a cost of **C**.
- **Step 2: Locate and Parse the Row.** The required row is located through the Array Pointer, incurring a cost **B**. Parsing the row to extract attributes adds a cost proportional to the number of attributes: $\text{Cost}_{\text{single_row}} = \mathbf{a}_{\text{row}} \times \mathbf{N}_{\text{attr}} + \mathbf{B}$, where $\mathbf{N}_{\text{attr}}$ is the number of attributes accessed.
- **Step 3: Retrieve Multiple Rows.** For multiple rows, the cost is: $\text{Cost}_{\text{multi_row}} = \mathbf{N}_{\text{rows}} \times \text{Cost}_{\text{single_row}}$, where $\mathbf{N}_{\text{rows}}$ is the number of rows accessed.
- **Step 4: Aggregate Costs Over Time.** Summing up costs for all queries gives: $\text{Cost}_{\text{row}} = \sum_{i=1}^{\mathbf{Q}} (\text{Cost}_{\text{multi_row}_i} + \mathbf{C})$, where **Q** is the total number of queries.

Next, let us examine the cost function for the column format. Since OLTP and OLAP have distinct access patterns in column format, they need to be discussed separately:

- **Step 1: Locate the BW-Tree Leaf Node.** Similar to the row format, the query first locates the relevant leaf node in the BW-Tree based on the query condition, represented by the constant **C**.
- **Step 2: OLTP Query in Column Format.** For OLTP queries, the access process in column format is similar to that in row format. However, due to differences in row and column storage, the cost of accessing each column attribute differs. We denote the cost of accessing each attribute in column format for OLTP as $\mathbf{a'}_{\text{col,oltp}}$. Thus, the cost function for multi-row OLTP queries in column format is $\text{Cost}_{\text{col,oltp,multi_row}} = \mathbf{N}_{\text{rows}} \times (\mathbf{B} + \mathbf{a'}_{\text{col,oltp}} \times \mathbf{N}_{\text{attr}})$, where $\mathbf{N}_{\text{rows}}$ represents the number of rows accessed, and $\mathbf{N}_{\text{attr}}$ represents the number of attributes accessed.
- **Step 3: OLAP Query in Column Format.** For OLAP queries, as described in Section 5.2.1, data from each column is scanned to retrieve values meeting the filtering criteria. The cost of scanning each column is proportional to the number of rows scanned. We define $\mathbf{a'}_{\text{col,olap}}$ as the average access cost per row for OLAP in column format. Thus, the cost of scanning each column is $\mathbf{a'}_{\text{col,olap}} \times \mathbf{N}_{\text{rows}}$. Since a query typically involves multiple columns, the total cost function for OLAP queries in column format is $\text{Cost}_{\text{col,olap,multi_col}} = \mathbf{a'}_{\text{col,olap}} \times \mathbf{N}_{\text{rows}} \times \mathbf{N}_{\text{attr}}$, where $\mathbf{N}_{\text{rows}}$ represents the number of rows scanned, and $\mathbf{N}_{\text{attr}}$ represents the number of attributes accessed.
- **Step 4: Aggregate Costs for OLTP and OLAP.** By summing up the costs for OLTP and OLAP queries over a given

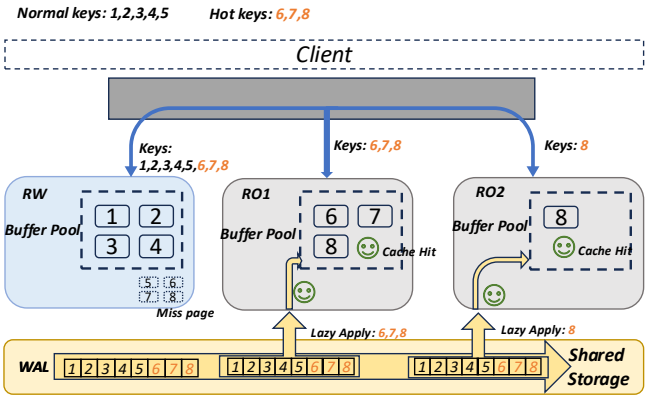


Figure 12: Hotspots Scaling Mechanism

time period, the total cost for column format is obtained, as shown in Equation 1.

To accurately define the parameters for online clusters, we selected a sample of datasets and measured execution times for different types of OLTP and OLAP queries as their costs. Using the sampled data, we calculated multiple values for $\mathbf{C}$, $\mathbf{B}$, $\mathbf{a}_{\text{row}}$, $\mathbf{a}'_{\text{col,oltp}}$, and $\mathbf{a}'_{\text{col,olap}}$, and computed their average values.

After evaluating the costs of row and column storage, we can determine the optimal storage format for an adjacency list. If the cost of column storage for handling numerous index-based scan queries significantly exceeds that of row storage, the format should be converted to row storage to optimize performance. Conversely, if frequent single-column queries cause the cost of row storage to exceed that of column storage, column storage becomes the more suitable choice. If the storage cost difference is negligible and the current format meets performance requirements, no conversion is necessary, avoiding unnecessary migration costs. The rules above keep per-list format choices both cheap to evaluate and easy to explain to operators.

The specific decision criteria are as follows:

- If the costs of both row and column storage exceed the threshold θ_{new}, both formats need to be maintained.
- If one format has a significantly higher cost than the other, and the cost difference exceeds the threshold θ_{convert}, the format should be converted to the one with the lower cost.

In the subsequent experimental section (5.4), we evaluate row-column conversions based on the existing cost function and compare them against pure row storage, pure column storage, and a hybrid row-column format. The results show that, while maintaining storage costs comparable to any single format, we achieve a performance improvement ranging from 11% to 70%. A tighter model could fit a linear regression over richer telemetry samples; we leave that data-driven calibration for future work.

4.4 Fine-grained Scaling Mechanism

To reduce replica maintenance costs under hotspot scenarios, we designed a dynamic mechanism that creates adjacency list–level replicas and distributes traffic based on hotspot awareness. Each node continuously monitors access patterns and reports hotspot information to a central control node. According to the observed hotspot intensity, the control node dynamically generates an appropriate number of read-only replicas using a lazy replay strategy—the higher the traffic, the more replicas are created. It then updates the routing policy so that hotspot requests are directed exclusively to these replicas, which only maintain pages related to the hotspot. This approach significantly lowers both maintenance cost and system complexity. To further enhance cache efficiency, the hotspot-aware traffic distribution mechanism balances load among the replay–based read-only nodes, ensuring that each node handles only the hotspot traffic it was designed to serve.

To better illustrate our approach, we assume a synthetic workload over 8 keys (1–8) to simulate temporal hotspot shifts. The workload evolves across three phases:

- **T_0–T_1:** Uniform access, all keys receive 10 QPS.
- **T_1–T_2:** Keys 1–5 maintain 10 QPS; keys 6–8 become hotspots with 200 QPS.
- **T_3–T_4:** Keys 1–5 remain at 10 QPS; keys 6–7 stay at 200 QPS; key 8 becomes a dominant hotspot with 800 QPS.

As shown in Figure 12, during the final phase (T_3–T_4), keys 6–8 become hotspots, with key 8 exhibiting the highest access frequency. To handle this evolving workload, node RO1 is assigned to serve requests for keys 6–8, while an additional read-only node, RO2, is dynamically created to exclusively handle the dominant hotspot key 8. This hotspot-aware traffic distribution sends each query to replicas that already hold the hot pages, so RO1 and RO2 both report 100% cache hits in this synthetic trace.

5 Evaluation

5.1 Experiment Setup

Workloads: We conducted our study using a real workload subset from the Douyin relationship scenario (as shown in Table 3). The dataset contains approximately 3 million vertices and 500 million edges, covering both OLTP and OLAP workloads. OLTP workloads primarily consist of single-hop exact queries, such as online queries for a user's Top-k followers or following lists. OLAP workloads involve multi-hop queries and path queries used for relationship mining and risk management. To account for workload fluctuations over a day, we selected three representative time periods: 1) **More OLTP**, where the OLTP and OLAP ratio is 3:1; 2) **More OLAP**, where the OLTP and OLAP ratio is 1:2; and 3) **OLTP & OLAP equal**, where the OLTP and OLAP ratio is 1:1. Across these time periods, the read/write ratio is approximately 95%/5%. To evaluate the impact of long-running OLAP transactions, we used OLAP-only and write traffic as a micro-benchmark during the "OLTP & OLAP equal" period.

Experiment Outline and Hardware Setup: We evaluated ByteGraph-Dione against ByteGraph [32] (the previous version) and two representative commercial graph database systems (denoted DB-X and DB-Y), under real workloads from the Douyin relationship scenario (Section 5.2). Due to scalability and storage capacity limitations in the Docker version of GalaxyBase, it was excluded from the comparison. Subsequently, we conducted a series

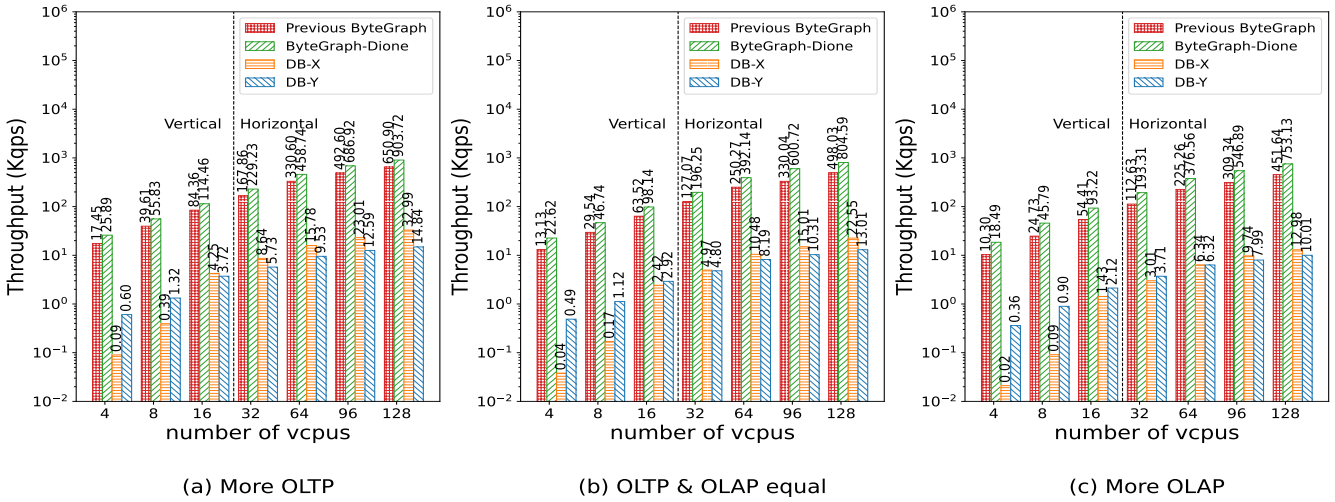


Figure 13: Log Transformation Overall Performance (Vertical: a single machine; Horizontal: 2 to 8 machines, each with 16 cores)

Workload	Description	OLTP/OLAP(Read Query)	Read/Write
More OLTP	Manage Douyin relational data records, including OLTP and OLAP hybrid workload. OLTP: One-hop query, such as TopK fan list, all follow list OLAP: multi-hop query, such as user relationship mining and relationship risk manager	3/1	95%/5%
More OLAP		1/2	98%/2%
OLTP & OLAP equal		1/1	97%/3%
Micro-benchmark OLAP-only		100% OLAP Read	90%/10%

Table 3: Workload Description.

of evaluations using real workloads or micro-benchmark tests to assess "Lightweight Transactions" (Section 5.3), "Adaptive Storage Format" (Section 5.4), and "Fine-grained Adjacency List Scaling Mechanism" (Section 5.5).

In the overall comparison experiment, DB-X and DB-Y were tested on an 8-node cluster; each node used an AWS `db.r5.4xlarge` instance (16 vCPUs, 128 GiB DRAM). Since previous ByteGraph and ByteGraph-Dione rely on internal append-only storage (similar to Alibaba Cloud's Pangu [22]), they were deployed on ByteDance's internal machines with equivalent performance to `db.r5.4xlarge`. All micro-benchmark tests were also conducted using nodes with the same specifications.

5.2 Overall Comparison

In this section, we use the hybrid workloads from the Douyin relationship scenario described in 5.1 to evaluate the performance of ByteGraph-Dione, previous ByteGraph [32], DB-X, and DB-Y. First, we assess the single-node scalability of ByteGraph-Dione, previous ByteGraph, DB-X, and DB-Y by increasing the number of available vCPU cores from 4 to 16. Next, we compare their performance across configurations scaled from 2 to 8 nodes, with each node equipped with 16 vCPU cores. As shown in Figure 13, ByteGraph-Dione demonstrates a significant performance advantage compared to DB-X and DB-Y, achieving throughput improvements of approximately 20X to 40X. When compared to previous ByteGraph, ByteGraph-Dione consistently outperforms it. In particular, in OLTP-dominant scenarios, ByteGraph-Dione achieves performance gains of approximately 35% to 50%, while in scenarios with balanced OLTP and OLAP workloads or OLAP-dominant workloads, ByteGraph-Dione achieves system throughput improvements of approximately 50%

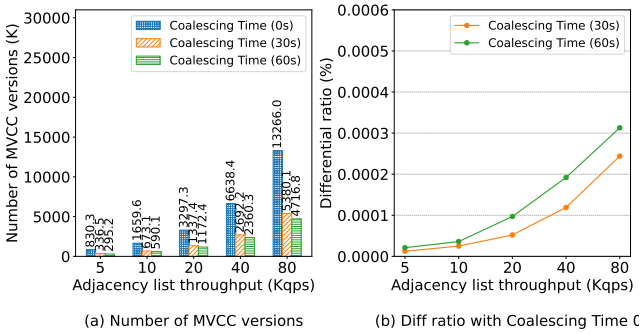


Figure 14: Evaluation under Different OLAP Throughput under Micro-benchmark OLAP-only (a) Comparison of the number of MVCC versions. (b) Analysis of the OLAP Result Discrepancies between the Proposed Approach and Traditional Methods.

to 80%. Adaptive row-column conversion lets each adjacency list pick the cheaper layout as traffic skew changes, which explains most of the gap over fixed-format engines.

5.3 Evaluation of Lightweight Transactions

5.3.1 Evaluation of OLAP Long-lived Transaction. As described in 4.2.1, by coalescing the version space for OLAP long-lived transactions, we significantly reduce the number of historical versions that need to be maintained. In this section, we conducted an experimental study using the "Micro-benchmark OLAP-only" workload described in Table 3. As shown in Figure 14(a), we precisely

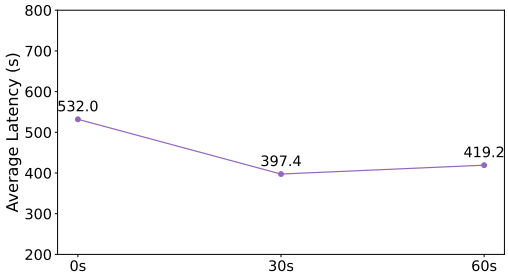


Figure 15: Latency Under Peak Load under Micro-benchmark OLAP-only

controlled the coalescing intervals and compared the number of historical versions maintained internally by ByteGraph-Dione under different OLAP throughput scenarios. The results demonstrate that, compared to no coalescing, a 30-second coalescing interval reduces the number of historical versions by 59.44%, while a 60-second coalescing interval achieves a reduction of 64.44%. Furthermore, as system throughput increases, coalescing trims version counts almost linearly with offered load.

Additionally, to evaluate the impact of coalescing on the accuracy of OLAP execution results, we conducted tests under different throughput conditions, comparing results for 30-second and 60-second coalescing intervals against non-coalesced execution. As shown in Figure 14(b), the differences in execution results were all less than 0.00035%, so coalescing barely shifts downstream analytics relative to a non-coalesced baseline.

To investigate the impact of coalescing interval on OLAP query execution performance, we monitored query execution latency under extreme load conditions with different coalescing interval configurations. As shown in Figure 15, setting the coalescing interval to 30 seconds resulted in the lowest overall query execution latency. Compared to a 30-second coalescing interval, a 60-second coalescing interval increased execution latency by approximately 5.48%. In contrast, without coalescing, execution latency significantly rose to 1.338 times that of the 30-second coalescing interval. Reducing live versions via coalescing therefore pays off despite a short warm-up phase before each batch. Longer 60-second windows add enough batching delay to offset part of that gain. In production we default to 30-second intervals because they offer the best latency trade-off in these tests.

5.3.2 Evaluation of Transaction Overhead. In this section, we used the "OLTP & OLAP equal" workload to evaluate the overhead of our proposed transaction mechanism. We compared the system's maximum throughput under three configurations: With RC, SI (Coalescing 0s), and SI (Coalescing 30s). As shown in Figure 16, our default setting, SI (Coalescing 30s), shows only a 16.49% throughput decrease compared to the With RC configuration. In contrast, traditional transaction implementations (SI (Coalescing 0s)) result in a 35.5% reduction in system throughput compared to With RC. These findings indicate that the combination of in-memory multi-versioning and the coalescing strategy effectively supports both short and long transactions, mitigating the negative impact of snapshot isolation (SI) on system throughput. Moreover, since roughly

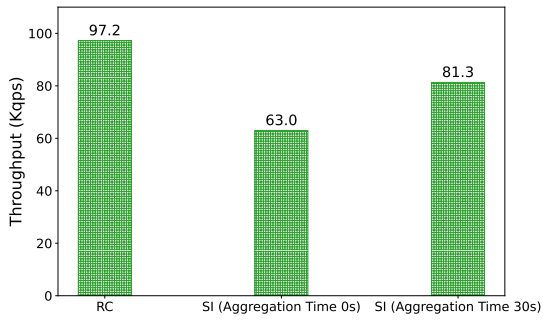


Figure 16: Throughput Comparison of With RC, SI (Coalescing 0s), and SI (Coalescing 30s) under OLTP & OLAP Equal Workload.

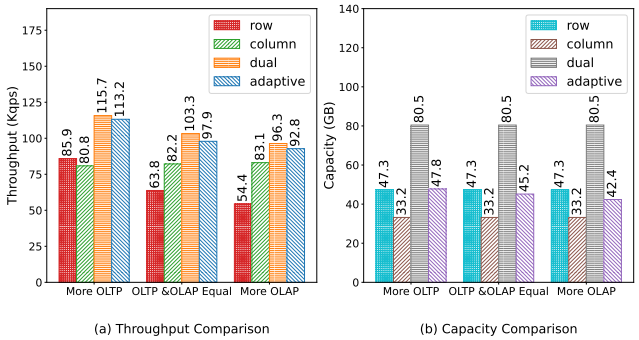
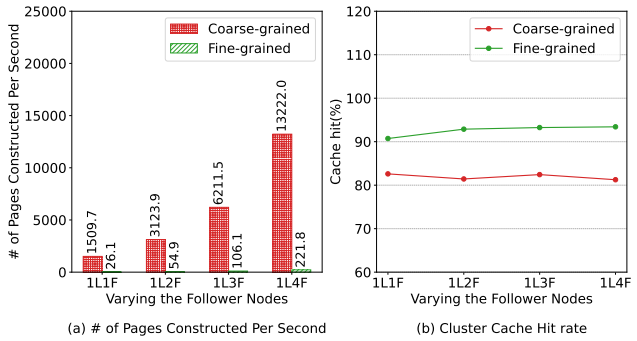


Figure 17: Throughput and Space occupation Comparison Among Different Storage Formats

half of the traffic in this mix still requires SI, shrinking that fraction would narrow the remaining gap to pure RC mode.

5.4 Evaluation on Adaptive Storage Format

5.4.1 Performance Evaluation. In this section, we evaluated the "Adaptive Storage Format" using the "More OLTP," "More OLAP," and "OLTP & OLAP equal" workloads. We assessed the maximum throughput performance of four storage modes: row storage, column storage, dual row-column storage, and the adaptive storage format. As shown in Figure 17(a), the adaptive storage format achieved performance improvements of approximately 31.8%, 53.5%, and 70.5% compared to row storage in the "More OLTP," "OLTP & OLAP equal," and "More OLAP" workloads, respectively. When compared to column storage, the adaptive storage format improved performance by about 40%, 19.1%, and 11.7% in the same workloads. The dual mode, which maintains redundant row and column formats to separately serve OLTP and OLAP workloads, achieves optimal performance but incurs roughly double the storage overhead compared to using only row or column storage. Our column storage maintains read-acceleration metadata to support fast OLTP reads, which limits the compression gain to about 30% over row storage in Figure 17(b). Our proposed adaptive storage format reduces storage overhead by approximately 41.25%, 45.75%, and 47.25% compared to the dual mode, thanks to the fine-grained dynamic row-column conversion module.



(a) # of Pages Constructed Per Second (b) Cluster Cache Hit rate

Figure 18: Page Construction & Cache Hit Rate Comparison under OLTP & OLAP Equal Workload.

5.4.2 Evaluation of Format Conversion Cost. As shown in columns "dual" and "adaptive" in Figure 17(a), their difference reflects the impact of row-column transformation on the overall system performance. ByteGraph-Dione's performance under the adaptive format was only slightly lower than the dual mode, with decreases of approximately 2.2%, 5.3%, and 3.7% in the respective workloads.

5.5 Evaluation on Fine-grained Scaling Mechanism

In this section, we evaluated the fine-grained scaling mechanism. For the experiment, we used the "OLTP & OLAP equal" workload and selected traffic from a single node, where hotspot traffic fluctuated between 10% and 80%. By varying the number of configured hotspot replicas, we compared the performance of ByteGraph-Dione and previous ByteGraph in terms of reconstructed page numbers and cache hit rates. As shown in Figure 18, ByteGraph-Dione reduced the number of reconstructed pages by approximately 98.32% compared to previous ByteGraph. With the number of follower nodes increasing from 1 to 4, ByteGraph-Dione reduced the number of reconstructed pages per second by 1484, 3069, 6105, and 13,000, respectively. The reduction in reconstructed pages is proportional to the increase in the number of nodes. Thanks to the fine-grained scaling mechanism, the number of pages constructed by ByteGraph-Dione on each node correlates only with the throughput of hotspot traffic. In terms of cache hit rates, as the number of follower nodes increased, ByteGraph-Dione 's cache hit rate improved from 90% to approximately 93%, while previous ByteGraph's cache hit rate remained around 82%. This improvement is attributed to ByteGraph-Dione 's hotspot-aware traffic distribution strategy, which directs hotspot data access to follower nodes, significantly enhancing cache utilization.

To investigate the overall system throughput under different hotspot proportions, we constructed a cluster consisting of three machines in a 1-master-2-follower configuration. With a fixed write traffic of 10,000, we adjusted the hotspot proportion to allow as many read requests as possible, thereby evaluating the maximum throughput under the OLTP & OLAP equal workload. We adopt the traditional coarse-grained hotspot replication approach as our baseline, replicating all data of the hotspot node whenever a hotspot occurs. Figure 19 shows that compared to coarse-grained, ByteGraph-Dione achieves a 3.39% throughput improvement when

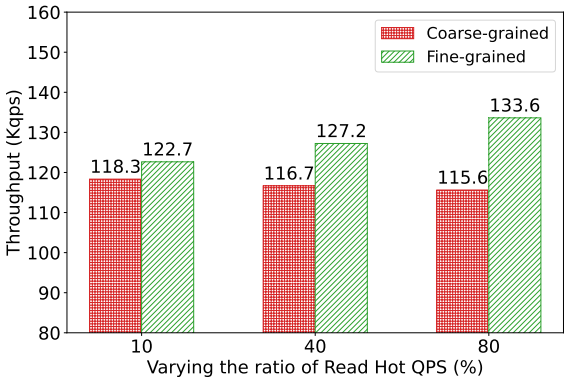


Figure 19: Throughput Comparison under Varying Hotspot Ratios

the hotspot traffic proportion is 10%. At 30% hotspot traffic, the throughput improvement increases to 9.48%, and at 80% hotspot traffic, the throughput improvement reaches 15.65%. Gains widen as hotspots concentrate because fine-grained followers rebuild far fewer cold pages while keeping working sets memory-resident on hot keys.

6 Conclusion

In this paper, we introduced ByteGraph-Dione, a high-performance graph database designed for ByteDance's dynamic hybrid workloads. ByteGraph-Dione employs a lightweight transaction mechanism to provide Snapshot Isolation for mixed OLTP/OLAP traffic with low overhead. Additionally, the system uses an adaptive dual storage format to speed up both point reads and wide scans as access patterns shift. Finally, it incorporates a fine-grained scaling mechanism to improve handling of sudden hotspot scenarios. Experiments on production-derived traces show ByteGraph-Dione beating prior ByteGraph and two commercial graph databases on the same hardware budget.

References

[1] 2021. JanusGraphg. https://janusgraph.org/.
[2] 2021. MySQL HeatWave. https://www.oracle.com/mysql/.
[3] 2022. MySQL. https://www.mysql.com/.
[4] 2022. PostgreSQL. https://www.postgresql.org/.
[5] 2023. Arangodb. https://arangodb.com/.
[6] 2023. orientdb. http://orientdb.org/.
[7] 2023. Titan. https://espeed.github.io/titandb/.
[8] 2024. Arrow Columnar Format. https://arrow.apache.org/docs/format/Columnar.html.
[9] Anastassia Ailamaki, David J DeWitt, Mark D Hill, and Marios Skounakis. 2001. Weaving Relations for Cache Performance.. In *VLDB*, Vol. 1. 169–180.
[10] Gökhan Aluç, M. Tamer Özsu, and Khuzaima Daudjee. 2014. Workload matters: Why RDF databases need a new design. *Proceedings of the VLDB Endowment* 7, 10 (2014), 837–840.
[11] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry Li, et al. 2013. {TAO}:{Facebook's} distributed data store for the social graph. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*. 49–60.
[12] Chiranjeeb Buragohain, Knut Magne Risvik, Paul Brett, Miguel Castro, Wonhee Cho, Joshua Cowhig, Nikolas Gloy, Karthik Kalyanaraman, Richendra Khanna, John Pao, et al. 2020. A1: A distributed in-memory graph database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 329–344.
[13] Jianjun Chen, Yonghua Ding, Ye Liu, Fangshi Li, Li Zhang, Mingyi Zhang, Kui Wei, Lixun Cao, Dan Zou, Yang Liu, et al. 2022. ByteHTAP: bytedance's HTAP

system with high data freshness and strong data consistency. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3411–3424.

[14] Alin Deutsch, Yu Xu, Mingxi Wu, and Victor Lee. 2019. Tigergraph: A native MPP graph database. *arXiv preprint arXiv:1901.08248* (2019).

[15] Franz Färber, Sang Kyun Cha, Jürgen Primsch, Christof Bornhövd, Stefan Sigg, and Wolfgang Lehner. 2012. SAP HANA database: data management for modern business applications. *ACM Sigmod Record* 40, 4 (2012), 45–51.

[16] Joseph E Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. {PowerGraph}: Distributed {Graph-Parallel} computation on natural graphs. In *10th USENIX symposium on operating systems design and implementation (OSDI 12)*. 17–30.

[17] Joseph E Gonzalez, Reynold S Xin, Ankur Dave, Daniel Crankshaw, Michael J Franklin, and Ion Stoica. 2014. {GraphX}: Graph processing in a distributed dataflow framework. In *11th USENIX symposium on operating systems design and implementation (OSDI 14)*. 599–613.

[18] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, et al. 2020. TiDB: a Raft-based HTAP database. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3072–3084.

[19] Tirthankar Lahiri, Shasank Chavan, Maria Colgan, Dinesh Das, Amit Ganesh, Mike Gleeson, Sanket Hase, Allison Holloway, Jesse Kamp, Teck-Hua Lee, et al. 2015. Oracle database in-memory: A dual format in-memory database. In *2015 IEEE 31st International Conference on Data Engineering*. IEEE, 1253–1258.

[20] Per-Åke Larson, Adrian Birka, Eric N Hanson, Weiyun Huang, Michal Nowakiewicz, and Vassilis Papadimos. 2015. Real-time analytical processing with SQL server. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1740–1751.

[21] Changji Li, Hongzhi Chen, Shuai Zhang, Yingqian Hu, Chao Chen, Zhenjie Zhang, Meng Li, Xiangchen Li, Dongqing Han, Xiaohui Chen, et al. 2022. ByteGraph: a high-performance distributed graph database in ByteDance. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3306–3318.

[22] Qiang Li, Qiao Xiang, Yuxin Wang, Haohao Song, Ridi Wen, Wenhui Yao, Yuanyuan Dong, Shuqi Zhao, Shuo Huang, Zhaosheng Zhu, et al. 2023. More than capacity: Performance-oriented evolution of pangu in alibaba. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. 331–346.

[23] Yucheng Low, Joseph E Gonzalez, Aapo Kyrola, Danny Bickson, Carlos E Guestrin, and Joseph Hellerstein. 2014. Graphlab: A new framework for parallel machine learning. *arXiv preprint arXiv:1408.2041* (2014).

[24] Grzegorz Malewicz, Matthew H Austern, Aart JC Bik, James C Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. 2010. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*. 135–146.

[25] Adam Prout, Szu-Po Wang, Joseph Victor, Zhou Sun, Yongzhu Li, Jack Chen, Evan Bergeron, Eric Hanson, Robert Walzer, Rodrigo Gomes, et al. 2022. Cloud-native transactions and analytics in singlestore. In *Proceedings of the 2022 International Conference on Management of Data*. 2340–2352.

[26] Tobias Schmidt, Dominik Durner, Viktor Leis, and Thomas Neumann. 2024. Two Birds With One Stone: Designing a Hybrid Cloud Storage Engine for HTAP. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3290–3303.

[27] Raghav Sethi, Martin Traverso, Dain Sundstrom, David Phillips, Wenlei Xie, Yutian Sun, Nezih Yegitbasi, Haozhun Jin, Eric Hwang, Nileema Shingte, et al. 2019. Presto: SQL on everything. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 1802–1813.

[28] Bing Tong, Yan Zhou, Chen Zhang, Jianheng Tang, Jing Tang, Leihong Yang, Qiye Li, Manwu Lin, Zhongxin Bao, Jia Li, et al. 2024. Galaxybase: A High Performance Native Distributed Graph Database for HTAP. *Proc. VLDB Endow* 17, 12 (2024), 3893–3905.

[29] Jianying Wang, Tongliang Li, Haoze Song, Xinjun Yang, Wenchao Zhou, Feifei Li, Baoyue Yan, Qianqian Wu, Yukun Liang, ChengJun Ying, et al. 2023. Polardb-imci: A cloud-native htap database system at alibaba. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25.

[30] Jingbo Xu, Zhanning Bai, Wenfei Fan, Longbin Lai, Xue Li, Zhao Li, Zhengping Qian, Lei Wang, Yanyan Wang, Wenyuan Yu, et al. 2021. Graphscope: A one-stop large graph processing system. *Proceedings of the VLDB Endowment* 14, 12 (2021), 2703–2706.

[31] Jiacheng Yang, Ian Rae, Jun Xu, Jeff Shute, Zhan Yuan, Kelvin Lau, Qiang Zeng, Xi Zhao, Jun Ma, Ziyang Chen, et al. 2020. F1 Lightning: HTAP as a Service. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3313–3325.

[32] Wei Zhang, Cheng Chen, Qiange Wang, Wei Wang, Shijiao Yang, Bingyu Zhou, Huiming Zhu, Chao Chen, Yongjun Zhao, Yingqian Hu, et al. 2024. BG3: A Cost Effective and I/O Efficient Graph Database in Bytedance. In *Companion of the 2024 International Conference on Management of Data*. 360–372.